

Guia Integral de la Entrega 4 del Proyecto Fin de Curso

Aplicación completa (Web y Móvil), calidad y evaluación experimental

Aplicaciones Distribuidas · Séptimo semestre · ISR-701

Universidad Técnica Estatal de Quevedo · Facultad de Ciencias de la Computación

Carrera: Ingeniería de Software

Asignatura: Aplicaciones Distribuidas (ISR-701)

Documento: Guia PFC Entrega 4 (E4) — Entrega final

Semana de entrega: Semana 17

Período: Regular 2026-2027

Modalidad: Grupal (equipos PFC de 3-4 integrantes)

Docente responsable: Ing. Gleiston C. Guerrero-Ulloa, Mgs.

Versión: v1.0

1. Presentación y lógica acumulativa de la entrega

La Entrega 4 (E4) constituye el cierre integral del PFC. En ella el sistema distribuido, construido incrementalmente durante todo el semestre, se completa con sus dos interfaces obligatorias (aplicación web y aplicación móvil), su capa de calidad (pruebas automatizadas y *pipeline* de integración y despliegue continuos), su infraestructura de observabilidad (métricas, registros y trazas distribuidas) y su evaluación experimental frente al modelo de calidad ISO/IEC 25010:2023 [1].

La E4 hereda directamente los artefactos producidos en las entregas anteriores: el análisis del dominio y el prototipo de comunicación de la E1, la arquitectura de microservicios REST con Docker Compose de la E2, y la capa de datos distribuida en CockroachDB con el pipeline paralelo PySpark de la E3. Ninguna de esas piezas se desecha. La E4 las envuelve, las refactoriza para elevar su cohesión y su calidad, y las expone a los usuarios finales a través de dos canales: un cliente web y un cliente móvil.

Requisito estricto de esta entrega. Cada equipo debe presentar dos aplicaciones cliente que consumen la misma API distribuida: (i) una aplicación web y (ii) una aplicación móvil. Ambas son obligatorias y no sustituibles entre sí. La ausencia total de una de ellas aplica un factor multiplicativo de 0,5 sobre la nota final de la entrega.

El nivel de rigor exigido corresponde al reporte de un experimento controlado en ingeniería de software: propósito, hipótesis, variables, protocolo, replicabilidad y discusión de amenazas a la validez [2, 3]. El manuscrito final consolidará todo el trabajo del semestre y será la evidencia única de evaluación del cierre.

2. Objetivos

2.1. Objetivo general

Completar, evaluar y documentar el sistema distribuido del PFC con arquitectura en capas, aplicaciones web y móvil, pipeline CI/CD, observabilidad distribuida y evaluación experimental de calidad, cumpliendo el modelo ISO/IEC 25010:2023 y los principios de reproducibilidad propios de la investigación empírica.

2.2. Objetivos específicos

1. Refactorizar el backend de E2–E3 aplicando arquitectura en capas, principios SOLID [4] y al menos cinco patrones de diseño GoF [5].
2. Desarrollar una aplicación web (SPA) que consume la API a través del API Gateway, con *routing*, gestión de estado, internacionalización y accesibilidad básica.
3. Desarrollar una aplicación móvil (Android nativa con Kotlin o multiplataforma con Flutter).

- r/React Native) que consume la misma API y hace uso de al menos dos capacidades del dispositivo (sensores, camara, notificaciones o geolocalizacion).
4. Establecer un *pipeline* completo de CI/CD con GitHub Actions que compila, prueba, empaqueta y publica artefactos de las tres capas (backend, web, móvil) [6, 7].
 5. Instrumentar el sistema con las tres señales de observabilidad (métricas, registros estructurados y trazas distribuidas) según la especificación OpenTelemetry [8, 9] y publicar un dashboard operativo.
 6. Evaluar el sistema contra cinco características del modelo ISO/IEC 25010:2023 con métricas objetivas medidas experimentalmente [1].
 7. Presentar el manuscrito final consolidando la totalidad del ciclo del PFC, con al menos 20 páginas de contenido, con estructura de reporte experimental y con análisis de amenazas a la validez.

3. Aplicación por equipo PFC

La Tabla 1 concreta la aplicación por equipo. Se supone conocida la arquitectura de microservicios de E2 y la capa de datos distribuida de E3; la E4 se centra en refactorizar el backend en capas, materializar los dos clientes y evaluar la calidad.

Tabla 1: Aplicación concreta de E4 por equipo PFC.

Equipo	Aplicación Web (SPA)	Aplicación Móvil (obligatoria)	Patrones GoF aplicados
AGLS TiendaTech	Catálogo, carrito, checkout, panel de vendedor, dashboard de métricas para el administrador.	Cliente móvil de compra: catálogo, carrito, notificaciones <i>push</i> de estado del pedido, cámara para reconocer códigos de barras.	Repository, Factory Method, Strategy (métodos de pago), Observer (eventos de pedido), Decorator (promociones).
BCEL SGA Escuela	Portal de docente, secretaría y directivo, con captura de calificaciones y reportes por período.	Cliente móvil para representantes: consulta de calificaciones, asistencia y comunicaciones, con notificaciones <i>push</i> .	Repository, Template Method (reportes), Observer (notificaciones), Facade (fachada del portal), Strategy (algoritmos de promedio).
FUVV Laboratorios	Panel de reserva y monitoreo con visualización de series temporales y calendario semanal.	Cliente móvil para técnicos: reservas rápidas, notificación de incidentes, escaneo de QR de sala.	Singleton (cliente Prometheus), Observer (alertas), State (estados de la reserva), Strategy (políticas de reserva), Facade.
ACC Soporte ISP	Consola de operadores y coordinadores, con vista de SLA por zona y por técnico.	Cliente móvil para técnicos de campo: tickets asignados, cierre en sitio, geolocalización y cámara para evidencias.	Command (acciones sobre tickets), Chain of Responsibility (escalado), Observer (SLA), Repository, Strategy (rutas).

4. Fundamento teórico exigible

4.1. Arquitectura en capas y principios SOLID

La arquitectura en capas separa el sistema en (i) presentación, (ii) aplicación, (iii) dominio e (iv) infraestructura [4, 10]. Los principios SOLID son las restricciones que impiden que las capas se acoplen incorrectamente y que las clases se conviertan en *objetos-Dios*: responsabilidad única,

abierto-cerrado, sustitución de Liskov, segregación de interfaz e inversión de dependencias. El uso disciplinado de estos principios es prerequisite para que los patrones GoF [5] aporten valor real.

4.2. Ingeniería de aplicaciones móviles

Las aplicaciones móviles se rigen por retos particulares: heterogeneidad de dispositivos, restricciones de batería y red, ciclo de vida gestionado por el sistema operativo y ventanas de publicación controladas por tiendas [11]. La decisión entre desarrollo nativo o multiplataforma no es una preferencia estética: tiene consecuencias medibles en rendimiento, mantenimiento y *time-to-market* [12, 13, 14]. El equipo debe justificar su elección con al menos dos criterios cuantitativos.

4.3. Integración y entrega continuas (CI/CD)

La entrega continua se apoya en un *pipeline* automatizado que ejecuta pruebas, empaqueta artefactos y despliega en ambientes segregados [7]. La cultura DevOps subyacente [6] demanda que ningún cambio llegue a producción sin pasar por los mismos controles. En el PFC el *pipeline* de GitHub Actions actúa como *quality gate*: si la integración falla, el despliegue no ocurre.

4.4. Observabilidad distribuida

La observabilidad de un sistema distribuido se sustenta en tres señales: métricas (agregados numéricos con etiquetas), registros (*logs* estructurados) y trazas (secuencias de *spans* correlacionados por un identificador común) [9, 15]. El estándar OpenTelemetry [8] unifica su modelo y su exportación, permitiendo que diferentes proveedores consuman las mismas señales. En el PFC estas tres señales se integran en un dashboard operativo que se demostrara en vivo.

4.5. Modelo de calidad ISO/IEC 25010:2023

El modelo ISO/IEC 25010:2023 [1] organiza la calidad de producto en ocho características: adecuación funcional, eficiencia de desempeño, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad. Cada característica se descompone en subcaracterísticas y admite medición mediante métricas objetivas. En el PFC se evaluarán cinco de ellas con al menos una métrica cuantitativa cada una.

5. Manual de desarrollo (paso a paso)

Se asume que el equipo parte de la rama `main` con la E3 fusionada. Todo el trabajo de esta entrega se realiza en la rama `feature/entrega-4` y se fusiona a `main` por *pull request* tras la revisión cruzada final.

5.1. Stack tecnológico consolidado

Stack obligatorio para E4

- **Backend:** Java 21 + Spring Boot 3.x (o el equivalente adoptado en E2), refactorizado en capas.
- **Persistencia distribuida:** cluster CockroachDB de la E3, sin cambios.
- **Analítica paralela:** pipeline PySpark de la E3, empaquetado en el mismo *compose*.
- **Web (SPA):** React 18 o Angular 17 (a elección del equipo, justificada); TypeScript; *router* declarativo; *state* con Redux Toolkit, Zustand, NgRx o Signals; *i18n* con *react-i18next* o *ngx-translate*; empaquetado en Docker *multi-stage* servido con Nginx.
- **Móvil:** Android nativa con Kotlin y Jetpack Compose, o multiplataforma con Flutter 3.x o React Native 0.74+.
- **CI/CD:** GitHub Actions con *matrix build* para las tres capas.
- **Observabilidad:** Prometheus (heredado de E2), Grafana, OpenTelemetry Collector, exportadores JVM y de sistema.
- **Pruebas de carga:** Locust 2.x.
- **Documentación:** LaTeX (repositorio ya lo tiene desde E1).

5.2. Módulo A: refactorización del backend en capas

Entrada: microservicio principal de E3 conectado al cluster CockroachDB.

Procedimiento:

1. Reorganizar el código en cuatro paquetes por microservicio: **presentation**, **application**, **domain** e **infrastructure**.
2. Extraer entidades y objetos de valor al paquete **domain**; en Spring Boot, estas entidades *no* deben tener anotaciones JPA (las anotaciones se colocan en clases de infraestructura).
3. Introducir puertos (interfaces) en **domain** y adaptadores en **infrastructure**: repositorios JDBC, clientes HTTP y *publishers* de mensajes.
4. Aplicar cinco patrones GoF con nombre y propósito documentados en un ADR (**docs/adr/ADR-005-patrones-gof.md**). La lista puede variar por equipo pero debe incluir al menos Repository, Factory Method y Strategy.
5. Actualizar las pruebas de E2 y E3 para que se apoyen en dobles de prueba de los puertos y no en la infraestructura real, exceptuando las pruebas de integración.

Salida esperada: el sistema mantiene el mismo comportamiento externo (API sin cambios), pero su estructura interna esta desacoplada. La cobertura de pruebas unitarias sube a $\geq 70\%$.

5.3. Módulo B: aplicación web (SPA)

Procedimiento (independiente del framework elegido):

1. Crear la carpeta **apps/web/** en el repositorio.
2. Configurar el proyecto con TypeScript en modo estricto y *linter* obligatorio (**eslint + prettier**).
3. Implementar el patrón de arquitectura por características (*feature-based*): cada módulo de negocio agrupa componentes, servicios de acceso a la API, hooks (o servicios) y pruebas.
4. Cubrir al menos cinco rutas: **/**, **/login**, **/main** (el dominio del PFC), **/settings** y **/about**; incluir manejo de rutas protegidas por JWT.
5. Integrar internacionalización (español e inglés como mínimo), tema claro/oscuro y estados de carga y de error explícitos.
6. Consumir la API del backend a través del API Gateway; guardar el JWT en **httpOnly cookie** o en **sessionStorage** con expiración, nunca en **localStorage** plano.

7. Empaquetar en *Dockerfile multi-stage* (etapa *builder* con Node LTS y etapa *runtime* con *nginx:alpine*); agregar el servicio al *docker-compose.yml*.
8. Publicar como *artefacto* en el pipeline CI un *.tar.gz* del *dist/* versionado con el hash de *commit*.

El Listado 1 muestra el Dockerfile de referencia.

```
1 # apps/web/Dockerfile
2 FROM node:20-alpine AS builder
3 WORKDIR /app
4 COPY package*.json ./
5 RUN npm ci
6 COPY . .
7 RUN npm run build
8
9 FROM nginx:1.27-alpine
10 COPY --from=builder /app/dist /usr/share/nginx/html
11 COPY nginx.conf /etc/nginx/conf.d/default.conf
12 EXPOSE 80
13 HEALTHCHECK --interval=30s --timeout=3s \
14   CMD wget -q -O- http://localhost/ || exit 1
15
```

Listing 1: Dockerfile multi-stage para la SPA.

5.4. Módulo C: aplicación móvil

Procedimiento (Android nativa como referencia; los equipos que elijan multiplataforma siguen los pasos analogos):

1. Crear *apps/mobile/* con proyecto Gradle Android; SDK mínimo API 26 (Android 8), objetivo API 34.
2. Definir la arquitectura MVVM con Jetpack Compose para la vista, ViewModel para el estado y un repositorio por característica.
3. Implementar la autenticacion consumiendo el mismo endpoint */auth/login* del backend; almacenar el JWT en *EncryptedSharedPreferences*.
4. Implementar el listado y el detalle del recurso principal del PFC, con *pull to refresh* y modo sin conexión (caché local con Room o SQLDelight).
5. Integrar al menos dos capacidades del dispositivo: notificaciones *push* (Firebase Cloud Messaging), camara (para OCR o códigos), geolocalizacion (fused location), o sensores de aceleración según el dominio del PFC.
6. Cubrir con pruebas unitarias los ViewModels (JUnit 5 + coroutines-test) y con al menos una prueba instrumentada *end-to-end* (Espresso o Compose Testing).
7. Generar un APK firmado con clave de *debug* y publicarlo como artefacto del pipeline CI en *release/apk/*.

5.5. Módulo D: pirámide de pruebas

La estrategia de pruebas cubre las cuatro capas de la pirámide clasica [16, 17]:

- **Unitarias:** cobertura $\geq 70\%$ en backend, web y móvil. Herramientas: JUnit 5 / Vitest o Jest / JUnit Android.
- **Integración:** pruebas de repositorios con Testcontainers apuntando a un CockroachDB efímero.
- **Contrato:** *consumer-driven contract testing* con Pact entre el frontend (web y móvil como consumidores) y el backend (proveedor).
- **Extremo a extremo (E2E):** Playwright para la web; Espresso o Detox para el móvil.

- **Carga:** Locust con al menos dos escenarios: (i) 50 usuarios durante 5 minutos, (ii) rampa progresiva de 0 a 200 usuarios en 10 minutos.

Cada tipo de prueba se ejecuta en un *job* distinto del pipeline y sus resultados se almacenan como artefactos.

5.6. Módulo E: pipeline CI/CD end-to-end

El archivo `.github/workflows/ci-cd.yml` define siete *jobs* en un *DAG*: `lint`, `test-backend`, `test-web`, `test-mobile`, `build-images`, `build-mobile-apk` e `integration`. Solo cuando todos los *jobs* estan en verde, el pipeline publica las imagenes en GitHub Container Registry (GHCR) etiquetadas con el hash del commit. El Listado 2 muestra un esqueleto reducido del pipeline.

```

1 name: pfc-cicd
2 on:
3   push:    { branches: [main, "feature/*"] }
4   pull_request:
5
6 jobs:
7   lint:
8     runs-on: ubuntu-latest
9     steps:
10      - uses: actions/checkout@v4
11      - name: Backend lint
12        run: ./mvnw -pl services/resource-service checkstyle:check
13      - name: Web lint
14        working-directory: apps/web
15        run: npm ci && npm run lint
16
17   test-backend:
18     needs: lint
19     runs-on: ubuntu-latest
20     steps:
21      - uses: actions/checkout@v4
22      - uses: actions/setup-java@v4
23        with: { distribution: temurin, java-version: "21" }
24      - run: ./mvnw -pl services/resource-service verify
25
26   test-web:
27     needs: lint
28     runs-on: ubuntu-latest
29     steps:
30      - uses: actions/checkout@v4
31      - working-directory: apps/web
32        run: |
33          npm ci
34          npm run test -- --coverage
35
36   test-mobile:
37     needs: lint
38     runs-on: ubuntu-latest
39     steps:
40      - uses: actions/checkout@v4
41      - uses: actions/setup-java@v4
42        with: { distribution: temurin, java-version: "17" }
43      - working-directory: apps/mobile
44        run: ./gradlew test
45
46   build-images:
47     needs: [test-backend, test-web]
48     runs-on: ubuntu-latest
49     permissions: { contents: read, packages: write }
50     steps:
51      - uses: actions/checkout@v4
52      - name: Login GHCR
53        uses: docker/login-action@v3
54        with:
55          registry: ghcr.io
56          username: ${GITHUB_ACTOR}
57          password: ${GITHUB_TOKEN}
58      - name: Build backend

```

```

59     run: docker build -t ghcr.io/${{ github.repository }}/backend:${{ github.sha }}
    ↪ services/resource-service
60     - name: Build web
61       run: docker build -t ghcr.io/${{ github.repository }}/web:${{ github.sha }} apps/web
62     - run: docker push --all-tags ghcr.io/${{ github.repository }}/backend
63     - run: docker push --all-tags ghcr.io/${{ github.repository }}/web
64
65 build-mobile-apk:
66   needs: test-mobile
67   runs-on: ubuntu-latest
68   steps:
69     - uses: actions/checkout@v4
70     - working-directory: apps/mobile
71       run: ./gradlew assembleDebug
72     - uses: actions/upload-artifact@v4
73       with:
74         name: pfc-debug-apk
75         path: apps/mobile/app/build/outputs/apk/debug/*.apk
76
77 integration:
78   needs: build-images
79   runs-on: ubuntu-latest
80   steps:
81     - uses: actions/checkout@v4
82     - run: docker compose -f docker-compose.yml up -d
83     - run: |
84       sleep 30
85       curl -f http://localhost:8080/actuator/health
86     - run: docker compose down
87

```

Listing 2: Esqueleto del pipeline `.github/workflows/ci-cd.yml`.

5.7. Módulo F: observabilidad distribuida

Procedimiento:

1. Añadir al microservicio principal el *starter* `opentelemetry-spring-boot-starter` (o equivalente).
2. Configurar el exportador OTLP hacia un *OpenTelemetry Collector* añadido al `docker-compose.yml`.
3. Reutilizar las métricas Prometheus de la E3 (`crdb_*`) y añadir cuatro nuevas: `http_requests_total` (etiquetado por ruta, método y código), `http_request_duration_seconds`, `app_business_events_total` y `app_active_sessions`.
4. Emitir registros en formato JSON con Logback (backend) y con Timber + tree JSON (móvil), incluyendo el campo `trace_id` para correlación.
5. Construir un dashboard Grafana con seis paneles obligatorios: tasa de peticiones, latencia p50/p95/p99, tasa de errores 5xx, disponibilidad Raft del cluster CRDB, uso de recursos por contenedor y latencia *end-to-end* desde el cliente móvil.
6. Exportar el dashboard como JSON versionado en `ops/grafana/pfc-dashboard.json` para que sea reproducible.

5.8. Módulo G: evaluación experimental ISO 25010

Protocolo experimental (rigor JCR):

1. **Sujeto de estudio:** el propio sistema del PFC en el *compose* de despliegue local.
2. **Instrumentos:** Locust para carga, Grafana para observación, scripts de pruebas contractuales y de disponibilidad.
3. **Diseño experimental:** bloque completo con $r = 10$ repeticiones por escenario; se descartan la primera y la última repetición.

4. **Métricas objetivas por característica ISO 25010** según la Tabla 2.
5. **Análisis:** media, desviación típica e intervalo de confianza al 95 %; se contrasta cada métrica con su umbral objetivo.
6. **Amenazas a la validez:** al menos tres internas y dos externas, discutidas en la sección de amenazas del manuscrito.

Tabla 2: Características ISO/IEC 25010:2023 evaluadas en E4 y sus métricas objetivas.

Característica	Métrica y protocolo de medición	Umbral objetivo
Fiabilidad	Tasa de errores 5xx durante una hora continua bajo carga nominal.	< 1 %
Eficiencia de desempeño	Latencia p95 con 50 usuarios concurrentes según escenario Locust.	< 500 ms
Seguridad	Porcentaje de endpoints con verificación JWT y ausencia de las 10 vulnerabilidades OWASP más comunes [18].	100 %
Mantenibilidad	Cobertura de pruebas unitarias + complejidad ciclomatica media.	$\geq 70\%$; < 10
Compatibilidad	Aplicación web funciona en Chrome, Firefox y Safari; aplicación móvil funciona en API 26+.	Pasa suite E2E

5.9. Módulo H: consolidacion del documento final

El manuscrito de la E4 es acumulativo: incluye, actualizadas, todas las secciones de E1, E2 y E3, y añade las secciones nuevas de U5. La cita a las entregas previas es interna: no se referencian como fuentes externas, sino que se integran en un texto continuo con estructura de reporte experimental. El total mínimo es de 20 páginas de contenido y ≥ 12 referencias IEEE con DOI o ISBN verificable.

6. Documento LaTeX exigido

La Tabla 3 lista la estructura mínima del manuscrito.

Tabla 3: Estructura mínima del manuscrito LaTeX de la Entrega 4.

Sección	Contenido	Páginas ref.
Resumen bilingue	Objetivo, método, resultados, conclusiones; ≤ 200 palabras cada uno.	1
Introducción	Motivación, contribuciones, panorama del semestre.	1.5
Trabajos relacionados	≥ 6 fuentes indexadas sobre BDD, microservicios, móvil y calidad.	2
Fundamento teórico	Capas + SOLID, GoF, móvil, CI/CD, observabilidad, ISO 25010.	3
Arquitectura del sistema	Vistas C4 nivel 1 a 3, decisiones (ADR), stack.	2
Aplicación web	Manual del Módulo B, capturas de las 5 rutas, evidencia i18n.	2
Aplicación móvil	Manual del Módulo C, capturas, evidencia de capacidades usadas.	2
Pruebas y CI/CD	Piramide de pruebas, resumen del pipeline, cobertura.	2
Observabilidad	Módulo F; dashboard como figura; ejemplo de traza distribuida.	1.5
Evaluación ISO 25010	Módulo G; tabla de métricas con IC 95 %; discusión.	2
Discusión y amenazas a la validez	Interpretación, alcance, limitaciones.	1.5
Conclusiones e trabajo futuro	Cierre del ciclo E1–E4.	1
Bibliografía	IEEE; ≥ 12 referencias verificadas.	—

7. Estructura del repositorio

El Listado 3 muestra la estructura resultante después del *merge* de la E4. Ninguna carpeta de las entregas anteriores desaparece; se añaden las de web, móvil y observabilidad.

```

1 proyecto-pfc/
2 README.md           # actualizado con instrucciones completas
3 LICENSE
4 .env.example         # incluye variables de web y móvil
5 docker-compose.yml   # backend + CRDB + web + collector
6 docs/
7   adr/
8     ADR-001-arquitectura.md
9     ADR-002-microservicios.md
10    ADR-003-fragmentación.md
11    ADR-004-consenso-raft.md
12    ADR-005-patrones-gof.md      # nuevo
13    ADR-006-elección-móvil.md    # nuevo
14  api/openapi.yaml
15  db/schema.sql
16  diagrams/
17    c4-nivel1.drawio.png
18    c4-nivel2.drawio.png
19    c4-nivel3.drawio.png        # actualizado con Web + Móvil
20  experimentos/
21    protocolo-e3.md
22    protocolo-e4.md             # nuevo
23  resultados/
24    iso25010.csv                # nuevo
25    boxplot_latencia.png        # nuevo
26  entrega1/ entrega2/ entrega3/ entrega4/
27  services/
28    resource-service/           # refactor en capas
29    auth-service/
30    notification-service/
31  apps/
32    web/                         # nuevo, SPA
33    mobile/                      # nuevo, Android/multiplataforma
34  spark/                         # heredado de E3
35  ops/
36    grafana/
37      pfc-dashboard.json         # nuevo
38    prometheus/
39      prometheus.yml             # actualizado
40      otel-collector/
41      config.yaml               # nuevo
42  release/
43    apk/                         # APK generado por CI
44    screenshots/                 # capturas de web y móvil
45  tests/
46    integration/
47    load/
48    e2e-web/                     # nuevo, Playwright
49    contract/                     # nuevo, Pact
50  .github/
51    workflows/
52      ci-cd.yml                  # ampliado con matriz web/móvil
53

```

Listing 3: Estructura del repositorio después de fusionar E4.

8. Rúbrica de evaluación

La rúbrica opera con nueve dimensiones, veintitun criterios y cinco niveles (0 a 4). Se aplica la fórmula

$$\text{Nota}_{/10} = \left[\frac{\sum_i w_i \cdot n_i}{4} \right] \cdot 10 \quad , \quad \sum_i w_i = 1.0 \quad (1)$$

Se recuerda que la ausencia total de la aplicación web o de la aplicación móvil aplica un factor multiplicativo de 0,5 sobre la nota final calculada con la Ecuación (1).

Tabla 4: Rúbrica de evaluación de la Entrega 4.

Dim.	Criterio	Indicador clave nivel 4	Peso
D1	1.1 Refactor en capas y SOLID	Cuatro capas, dependencias del dominio hacia abstracciones	6 %
	1.2 Patrones GoF aplicados	≥ 5 patrones justificados en ADR-005; código real, no placebo	6 %
D2	2.1 Aplicación Web (funcional)	Cinco rutas, i18n, tema, JWT correcto, sin errores de consola	8 %
	2.2 Aplicación Web (calidad)	Lint verde, cobertura ≥ 70 %, Dockerfile multi-stage	4 %
D3	3.1 Aplicación Móvil (funcional)	Login, listado y detalle, dos capacidades del dispositivo	8 %
	3.2 Aplicación Móvil (calidad)	Tests unit + instrumentados; APK firmado publicado por CI	4 %
D4	4.1 Integración Web + Móvil + Backend	Contratos Pact en verde; misma API sirve a los dos clientes	5 %
	4.2 Persistencia distribuida operando	Cluster CRDB de E3 sigue funcionando; healthchecks verdes	2 %
D5	5.1 Piramide de pruebas	Unit + Integración + Contrato + E2E + Carga; artefactos publicados	6 %
	5.2 Pipeline CI/CD end-to-end	Los 7 jobs del Listado 2 en verde; imagenes en GHCR	5 %
D6	6.1 Observabilidad completa	Métricas + logs con trace_id + trazas OTel; dashboard versionado	6 %
	6.2 Evidencia operativa	Demo en vivo del dashboard bajo carga Locust en la defensa	2 %
D7	7.1 Evaluación ISO 25010	Cinco características medidas con IC 95 %; discusión honesta	5 %
D8	8.1 Manuscrito final	Estructura de Tabla 3; ≥ 20 páginas; sin errores LaTeX	5 %
	8.2 Bibliografía	IEEE, ≥ 12 referencias, la mayoría con DOI resoluble	3 %
	8.3 Trazabilidad E1–E4	Se muestra que los artefactos de las entregas previas siguen vivos	3 %
	8.4 Repositorio y README	README completo, <code>.env.example</code> vigente, semillas deterministas	3 %
D9	9.1 Defensa oral (grupal)	Demo en vivo; cada integrante responde preguntas sin leer	5 %
	9.2 Reflexión ética	$\geq$ media página con referencia al ACM Code of Ethics [19]	2 %
	9.3 Discusión de amenazas a la validez	Al menos tres internas y dos externas, cada una con mitigación	2 %
	9.4 Reproducibilidad	<code>dockercomposeup</code> + instrucciones despliegan todo desde cero	2 %

9. Anexos

Anexo A. Lista de verificación final

- ☐ Rama `feature/entrega-4` fusionada a `main` por PR con revisión cruzada.
- ☐ Backend refactorizado en capas; ADR-005 firmado.
- ☐ Aplicación web (SPA) desplegada con `docker compose up`; ≥ 5 rutas; i18n.

- ☐ Aplicación móvil compilando; APK generado por CI; dos capacidades del dispositivo.
- ☐ Pipeline CI/CD con los 7 jobs verdes en el último commit de la entrega.
- ☐ Contratos Pact publicados y verdes; suite E2E Playwright y Espresso pasando.
- ☐ Prueba de carga Locust ejecutada; capturas incluidas en el manuscrito.
- ☐ Dashboard Grafana operativo con las seis paneles obligatorios.
- ☐ Tabla ISO 25010 con las cinco características y sus IC 95 %.
- ☐ Manuscrito LaTeX compila sin errores; ≥ 20 páginas de contenido.
- ☐ Bibliografía con ≥ 12 referencias IEEE verificadas.
- ☐ Reflexión ética de al menos media página.
- ☐ README con instrucciones de despliegue reproducible.

Anexo B. Preguntas obligatorias para la defensa

1. Justifique la elección de framework para la web y para el móvil, con al menos dos criterios cuantitativos.
2. Explique como los principios SOLID se materializan en el módulo más complejo de su backend.
3. Muestre una traza distribuida completa desde la app móvil hasta el cluster CRDB y describa cada *span*.
4. Explique como el pipeline CI/CD garantiza que el código desplegado cumple los atributos de calidad definidos.
5. Interprete los resultados de la evaluación ISO 25010: que característica no alcanzo el umbral y por que.
6. Describa el plan técnico para llevar el sistema a producción real, incluyendo escalado horizontal.
7. Discuta al menos tres amenazas a la validez de su evaluación experimental y como las mitigo.
8. Reflexione sobre las responsabilidades eticas del equipo frente a los datos de los usuarios finales.

Anexo C. Plantilla de ADR (formato Nygard)

Cada ADR (*Architectural Decision Record*) debe seguir el formato:

- **Título:** número y título corto.
- **Estado:** propuesto, aceptado, sustituido, deprecado.
- **Contexto:** problema, restricciones y alternativas consideradas.
- **Decisión:** elección tomada y razones.
- **Consecuencias:** positivas, negativas y neutrales, con impacto en las entregas anteriores y futuras.

Referencias

- [1] International Organization for Standardization, “ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (SQuaRE) — product quality model.” ISO Standard, 2023.
- [2] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.
- [3] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

- [4] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ, USA: Prentice Hall, 2017.
- [5] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.
- [6] G. Kim, J. Humble, P. Debois, and J. Willis, *The DevOps Handbook*. Portland, OR, USA: IT Revolution Press, 2 ed., 2021.
- [7] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston, MA, USA: Addison-Wesley, 2010.
- [8] OpenTelemetry Authors, "OpenTelemetry specification, version 1.32." CNCF Sandbox Project Documentation, 2024.
- [9] B. H. Sigelman, L. A. Barroso, M. Burrows, P. Stephenson, M. Plakal, D. Beaver, S. Jaspan, and C. Shanbhag, "Dapper, a large-scale distributed systems tracing infrastructure," Tech. Rep. dapper-2010-1, Google, Inc., 2010.
- [10] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA, USA: Addison-Wesley, 2003.
- [11] A. I. Wasserman, "Software engineering issues for mobile application development," in *Proc. FSE/SDP Workshop on Future of Software Engineering Research (FoSER)*, pp. 397–400, 2010.
- [12] W. S. El-Kassas, B. A. Abdullah, A. H. Yousef, and A. M. Wahba, "Taxonomy of cross-platform mobile applications development approaches," *Ain Shams Engineering Journal*, vol. 8, no. 2, pp. 163–190, 2017.
- [13] A. Biørn-Hansen, T.-M. Grønli, G. Ghinea, and S. Alouneh, "An empirical study of cross-platform mobile development in industry," *Wireless Communications and Mobile Computing*, vol. 2019, p. 5743892, 2019.
- [14] A. Ahmad, K. Li, C. Feng, S. M. Asim, A. Yousif, and S. Ge, "An empirical study of investigating mobile applications development challenges," *IEEE Access*, vol. 6, pp. 17711–17728, 2018.
- [15] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA, USA: O'Reilly Media, 2016.
- [16] P. Ammann and J. Offutt, *Introduction to Software Testing*. Cambridge, UK: Cambridge University Press, 2 ed., 2016.
- [17] T. Winters, T. Manshreck, and H. Wright, eds., *Software Engineering at Google: Lessons Learned from Programming over Time*. Sebastopol, CA, USA: O'Reilly Media, 2020.
- [18] OWASP Foundation, "OWASP top 10 web application security risks — 2021." Online report, 2021.
- [19] ACM Council, "ACM code of ethics and professional conduct." Online policy document, 2018.